

Chapter 4.2

Network Operations

Contents

Mobile-to-server communication

Server Options

Networking

Network Resources

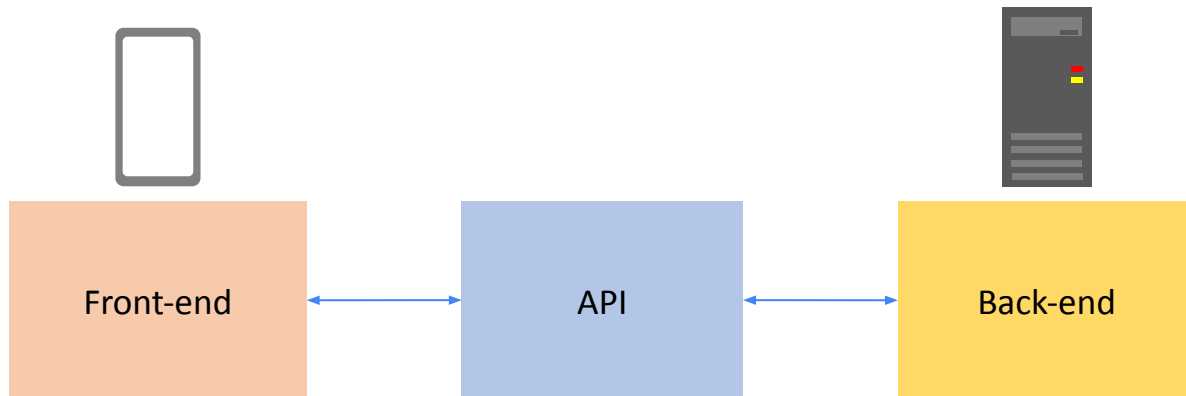
Message Queuing Telemetry Transport

Mobile-to-server communication

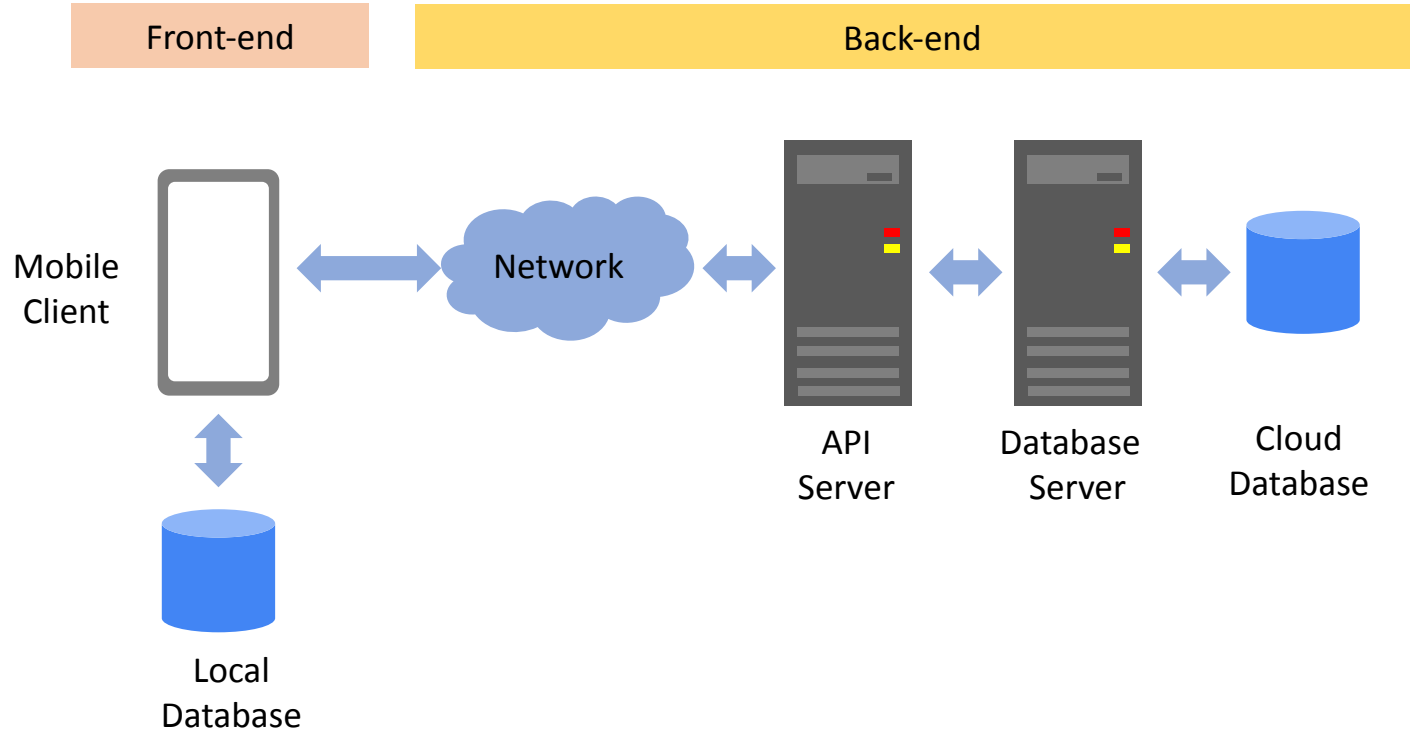
Mobile-to-server Communication

Most mobile apps are the front-end interfaces of back-end services

The two components use Application Programming Interface (API) to communicate with each other



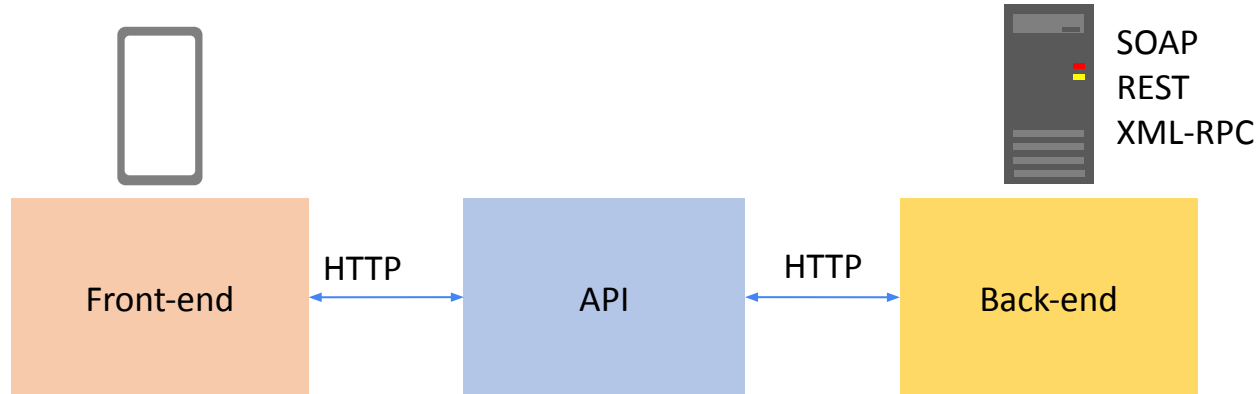
Mobile System Architecture



Communication Protocol

HTTP is the most commonly used protocol for communication

Methods of communication: SOAP, REST, and XML-RPC



Machine-to-machine Communication

Representational State Transfer ([REST](#))

- Supports data formats: HTML, XML and JavaScript Object Notation ([JSON](#))
- Inherits HTTP operations; GET, POST, PUT and DELETE

<https://www.restapitutorial.com/lessons/whatisrest.html>

Machine-to-machine Communication

Simple Object Access Protocol ([SOAP](#))

- Supports data formats: XML
- Works with application layer protocol. E.g. HTTP, SMTP, TCP, or UDP

REST data formats

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<authentication-context>
  <username>my_username</username>
  <password>my_password</password>
  <validation-factors>
    <validation-factor>
      <name>remote_address</name>
      <value>127.0.0.1</value>
    </validation-factor>
  </validation-factors>
</authentication-context>
```

JSON

```
{
  "username" : "my_username",
  "password" : "my_password",
  "validation-factors" : {
    "validationFactors" : [ {
      "name" : "remote_address",
      "value" : "127.0.0.1"
    } ]
  }
}
```

SOAP data formats

```
<env:Envelope
xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header>
    <n:alertcontrol xmlns:n="http://example.org/alertcontrol">
      <n:priority>1</n:priority>
      <n:expires>2001-06-22T14:00:00-05:00</n:expires>
    </n:alertcontrol>
  </env:Header>
  <env:Body>
    <m:alert xmlns:m="http://example.org/alert">
      <m:msg>Pick up Mary at school at 2pm</m:msg>
    </m:alert>
  </env:Body>
</env:Envelope>
```

Server Options

Server Options

DIY

Build
your own
servers



Subscribe

Back-End
as a
Service
(BaaS)



Mix and bang

DIY +
Subscribe



Server Options

Factors to Consider:

1. Project Scope: How complex is your app? How much data will it handle?
2. Budget: What is your budget for development and ongoing maintenance?
3. Team Expertise: Do you have in-house expertise in server-side development?
4. Scalability Requirements: How much growth do you anticipate for your app?
5. Time-to-Market: How quickly do you need to launch your app?

Server Options - DIY (Do It Yourself)



Advantages:

You have complete control over the technology stack, data, and infrastructure.

It can become cost-effective in the long run if you manage it well.

You earn valuable experience in server-side development.

Server Options - DIY (Do It Yourself)



Disadvantages:

High initial investment for setup, maintenance, and scaling.

Expertise required in server administration, networking, security, and database management.

Requires ongoing maintenance and troubleshooting.

Server Options - Subscribe



Advantages:

No need to manage servers, databases, or infrastructure.
Gain access to pre-build features such as AI model, NLP etc

Providers handle scaling and performance automatically.
Allows you to focus on building your app's frontend and core functionality.

Fast and cost-effective deployment - you can introduce your services to the public quickly.

Server Options - Subscribe



Disadvantages:

Vendor Lock-in. It can be challenging to switch providers later.

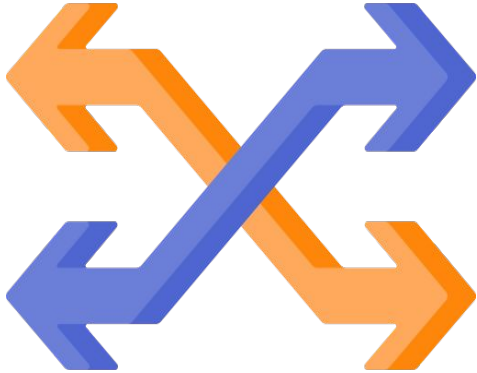
May have limitations on customization and flexibility.

Costs can increase as your app grows and usage increases.

Server Options - Mix and bang

Advantages:

It is flexible. Combines the benefits of DIY and BaaS.



Customizable: Allows you to choose and manage specific components while relying on BaaS for others.

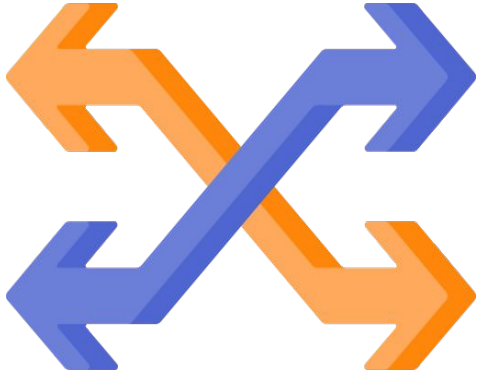
Improved Scalability: Can scale specific components as needed.

Server Options - Mix and bang

Disadvantages:

Increased Complexity: Requires careful planning and coordination.

Requires a deeper understanding of both approaches.



Networking

Networking

Flutter uses a cross-platform http networking

The `http` package provides the simplest way to issue http requests.

This package is supported on Android, iOS, macOS, Windows, Linux and the web.

Networking

Android

Android apps must declare their use of the internet in the Android manifest (AndroidManifest.xml):

```
<manifest xmlns:android...>
  ...
  <uses-permission android:name="android.permission.INTERNET" />
  <application ...
</manifest>
```

Networking

macOS

macOS apps must allow network access in the relevant *.entitlements files.

```
<key>com.apple.security.network.client</key>  
<true/>
```

Network Resources

Getting file from the network

```
import 'package:flutter/material.dart';  
import 'package:http/http.dart' as http;  
import 'dart:convert';
```

```
// Getting a file from the network
```

```
class NetworkImageExample extends StatelessWidget {  
  final String imageUrl = 'https://example.com/image.jpg';
```

```
@override
```

```
Widget build(BuildContext context) {  
  return FutureBuilder<http.Response>(  
    future: http.get(Uri.parse(imageUrl)),  
    builder: (context, snapshot) {  
      if (snapshot.hasData) {  
        final bytes = snapshot.data!.bodyBytes;  
        return Image.memory(bytes);  
      } else if (snapshot.hasError) {  
        return Text('Error loading image');  
      } else {  
        return const CircularProgressIndicator();  
      }  
    },  
  );  
}
```

Network Operations

Network Operations

Flutter provides various methods to perform network operations, primarily through the `http` package.

This package allows you to make HTTP requests to interact with web APIs, such as:

- GET
- POST
- PUT
- DELETE

Network Operations

GET

Retrieves data from a specified resource.

Typically used for fetching information, such as news feeds, product listings, or user profiles.

```
http.get(Uri.parse('https://api.example.com/users'))  
    .then((response) {  
        // Handle the response  
    });
```

Network Operations

POST

Sends data to a server to create a new resource.

Often used for submitting forms, uploading files, or creating new records in a database.

```
http.post(Uri.parse('https://api.example.com/users'), body: {  
  'name': 'John Doe',  
  'email': 'johndoe@example.com'  
}).then((response) {  
  // Handle the response  
});
```

Network Operations

PUT

Updates an existing resource on the server.

Typically used to modify existing data or replace an entire resource.

```
http.put(Uri.parse('https://api.example.com/users/123'), body: {  
  'name': 'Jane Doe'  
}).then((response) {  
  // Handle the response  
});
```

Network Operations

DELETE

Removes a resource from the server.

Used to delete existing records or resources.

```
http.delete(Uri.parse('https://api.example.com/users/123'))  
    .then((response) {  
    // Handle the response  
});
```


Fetch data from the internet

This recipe uses the following steps:

1. Add the http package.
2. Make a network request using the http package.
3. Convert the response into a custom Dart object.
4. Fetch and display the data with Flutter.

Fetch data from the internet

Step 1: Add the http package

The http package provides the simplest way to fetch data from the internet.

To add the http package as a dependency, run

```
$flutter pub add
```

Or

Add the http package to your pubspec.yaml file:

```
dependencies:
```

```
  http: ^0.13.5
```

Fetch data from the internet

Step 1: Add the http package

Import the http package in your code:

```
import 'package:http/http.dart' as http;
```

Fetch data from the internet

Step 1: Add the http package

Android

Edit your `AndroidManifest.xml` file to add the Internet permission.

```
<manifest xmlns:android...>
  ...
  <uses-permission android:name="android.permission.INTERNET" />
  <application ...
</manifest>
```

Fetch data from the internet

Step 1: Add the http package

iOS

Edit your macos/Runner/DebugProfile.entitlements and macos/Runner/Release.entitlements files to include the network client entitlement.

```
<key>com.apple.security.network.client</key>  
<true/>
```

Fetch data from the internet

Step 2: Make a network request using the http package

Use the `http.get()` method to retrieve data from a server

```
Future<http.Response> fetchAlbum() {  
    return http.get(Uri.parse('https://jsonplaceholder.typicode.com/albums/1'));  
}
```

The `http.Response` class contains the data received from a successful http call.

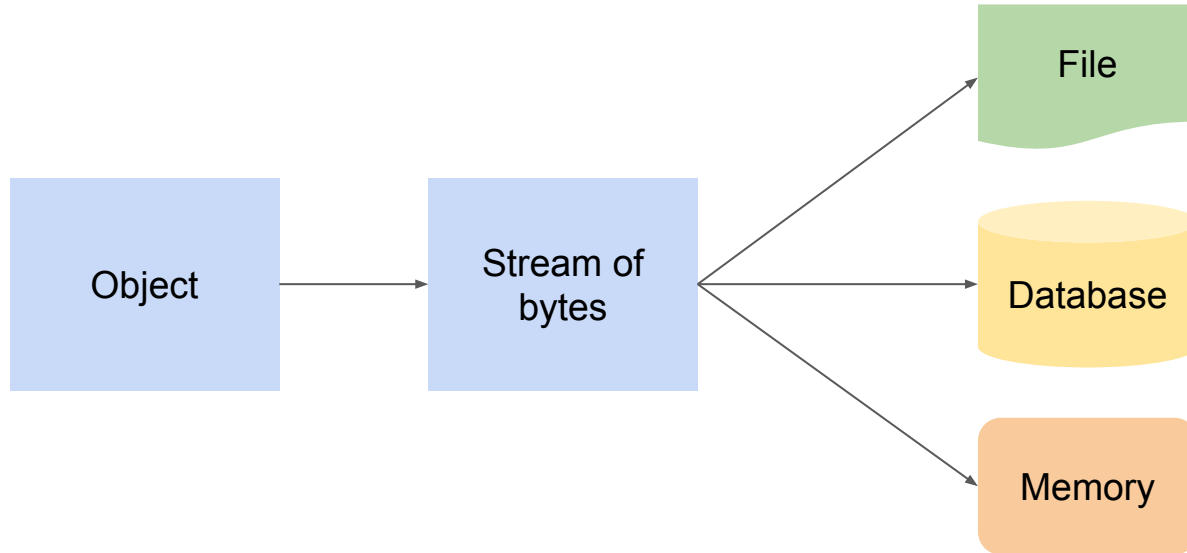
JSON

JSON is an open standard file format and data interchange format that uses human-readable text to store and transmit data objects consisting of attribute–value pairs and arrays.

It is a commonly used data format with diverse uses in electronic data interchange, including that of web and mobile applications with servers.

Serialization

Serialization refers to the process of translating data structures to and from a more easily readable format.



JSON Serialization

Flutter offers two general strategies for working with JSON:

1. Manual serialization
2. Automated serialization using code generation

Manual Serialization

It uses the built-in JSON decoder in `dart:convert`.

It involves passing the raw JSON string to the `jsonDecode()` function, and then looking up the values you need in the resulting `Map<String, dynamic>`.

Manual Serialization

Advantages:

- It has no external dependencies or particular setup process
- It is good for a quick proof of concept.

Manual Serialization

Disadvantages:

- It does not perform well when your project becomes bigger.
- Writing decoding logic by hand can become hard to manage and error-prone.
- If you have a typo when accessing a nonexistent JSON field, your code throws an error during runtime.

Code generation

Uses an external library generate the encoding boilerplate. E.g. `json_serializable` and `built_value` libraries.

Advantages:

- It scales well for a larger project.
- No hand-written boilerplate is needed, and typos when accessing JSON fields are caught at compile-time.

Code generation

Disadvantages:

- It requires some initial setup
- The generated source files might produce visual clutter in your project navigator.

Serializing JSON inline

```
{  
  "name": "John Smith",  
  "email": "john@example.com"  
}
```

```
final user = jsonDecode(jsonString) as Map<String, dynamic>;
```

```
print('Howdy, ${user['name']}!');
```

```
print('We sent the verification link to ${user['email']}.');
```

Serializing JSON inside model classes

With this approach, the calling code can have type safety, autocompletion for the name and email fields, and compile-time exceptions.

```
class User {  
  final String name;  
  final String email;  
  
  User(this.name, this.email);  
  
  User.fromJson(Map<String, dynamic> json)  
    : name = json['name'] as String,  
      email = json['email'] as String;  
  
  Map<String, dynamic> toJson() => {  
    'name': name,  
    'email': email,  
  };  
}
```


Serializing JSON inside model classes

The responsibility of the decoding logic is now moved inside the model itself. With this new approach, you can decode a user easily.

```
final userMap = jsonDecode(jsonString) as Map<String, dynamic>;  
final user = User.fromJson(userMap);  
  
print('Howdy, ${user.name}!');  
print('We sent the verification link to ${user.email}.');
```

Serializing JSON inside model classes

To encode a user, pass the `User` object to the `jsonEncode()` function. You don't need to call the `toJson()` method, since `jsonEncode()` already does it for you.

```
String json = jsonEncode(user);
```

Message Queuing Telemetry Transport

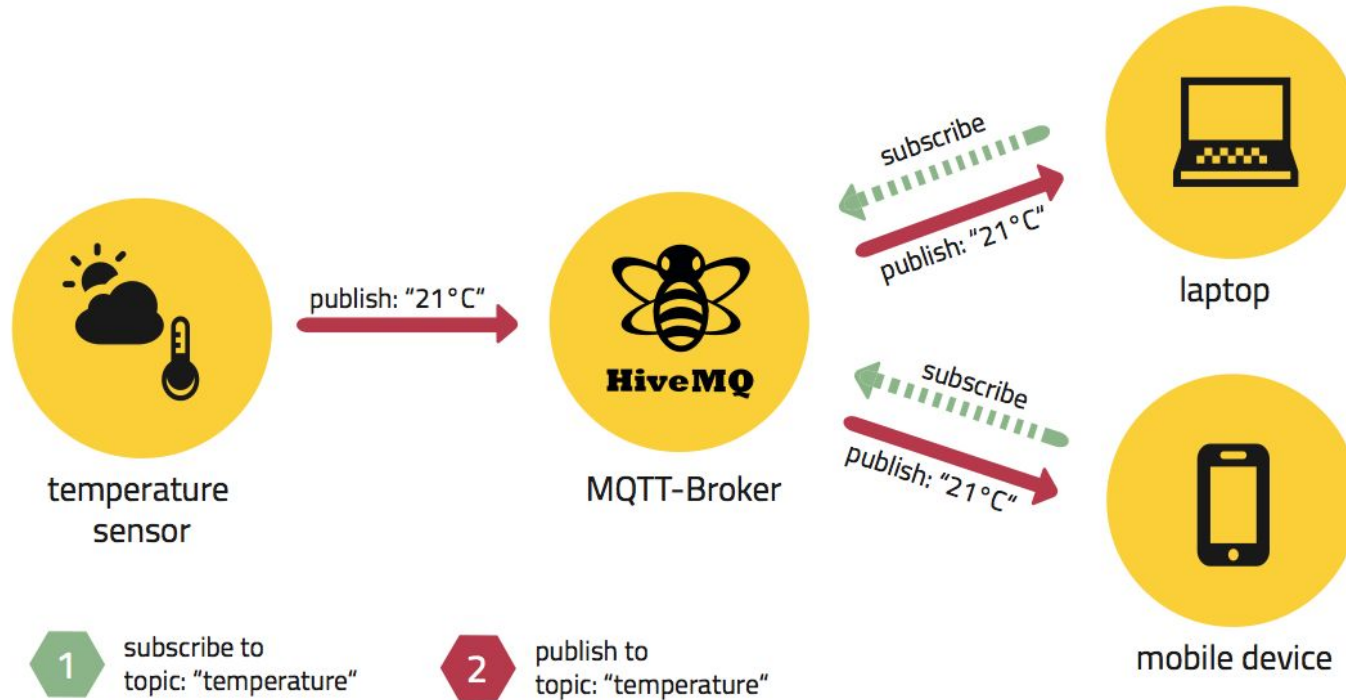
Message Queuing Telemetry Transport ([MQTT](#))

Works on top of TCP/IP

Machine-to-machine connectivity protocol

Uses publish/subscribe messaging transport

Internet of Things Real-time messaging



Reference

Networking, <https://docs.flutter.dev/data-and-backend/networking>

JSON and serialization,

<https://docs.flutter.dev/data-and-backend/serialization/json>

Fetch data from the internet,

<https://docs.flutter.dev/cookbook/networking/fetch-data>

Find out more

Introduction to MQTT, <https://github.com/mqtt/mqtt.github.io/wiki>

MQTT Client, https://pub.dev/packages/mqtt_client